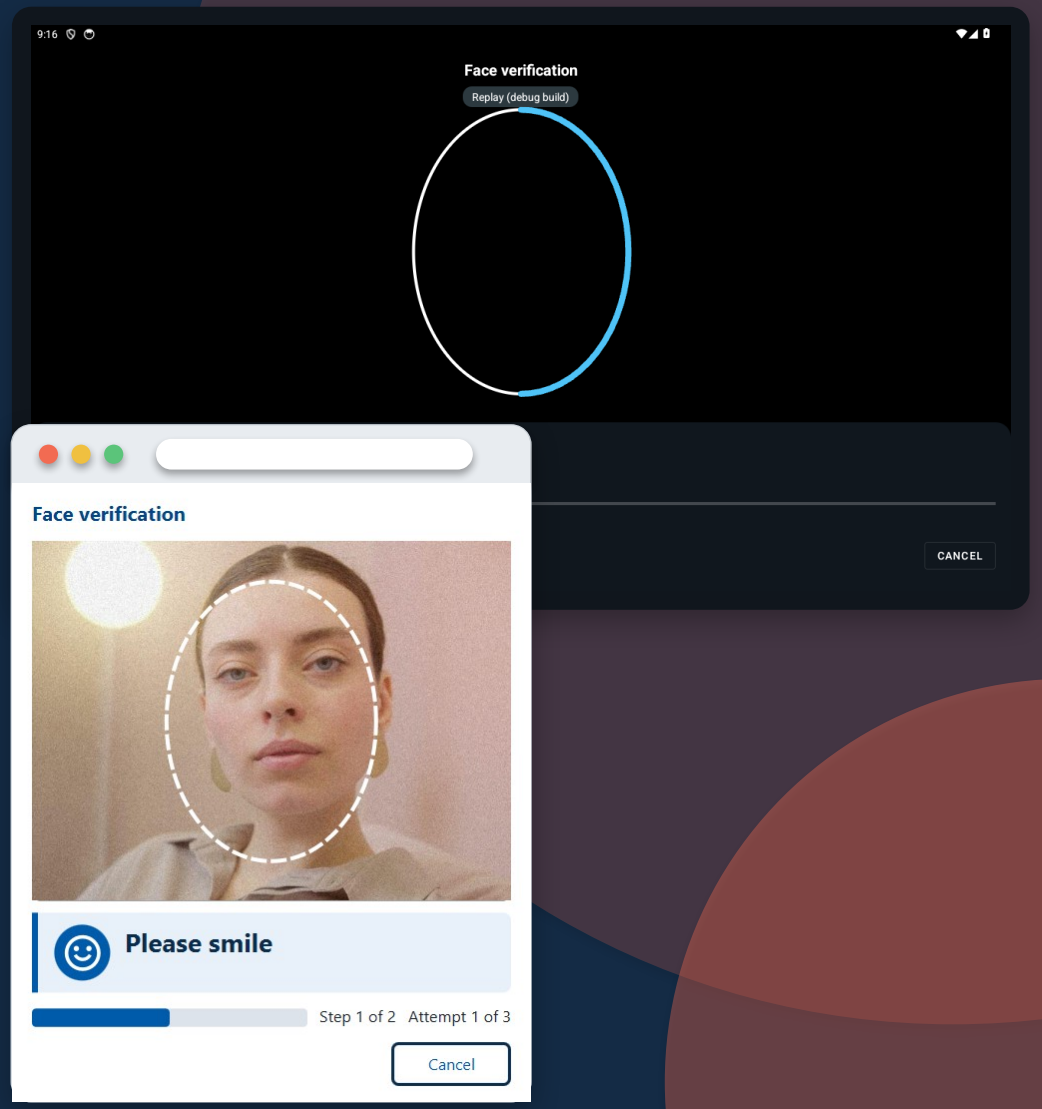
MOSIP DECODE 2026 · PROBLEM STATEMENT 04

Face liveness and presentation attack detection

For both MOSIP registration clients: offline, vendor-neutral, configurable per workflow



Team Badmosip

Kanav Bhardwaj · Ivan Bhargava · Nikunj Mahajan
Ishaan Sodhi · Harsh Modi

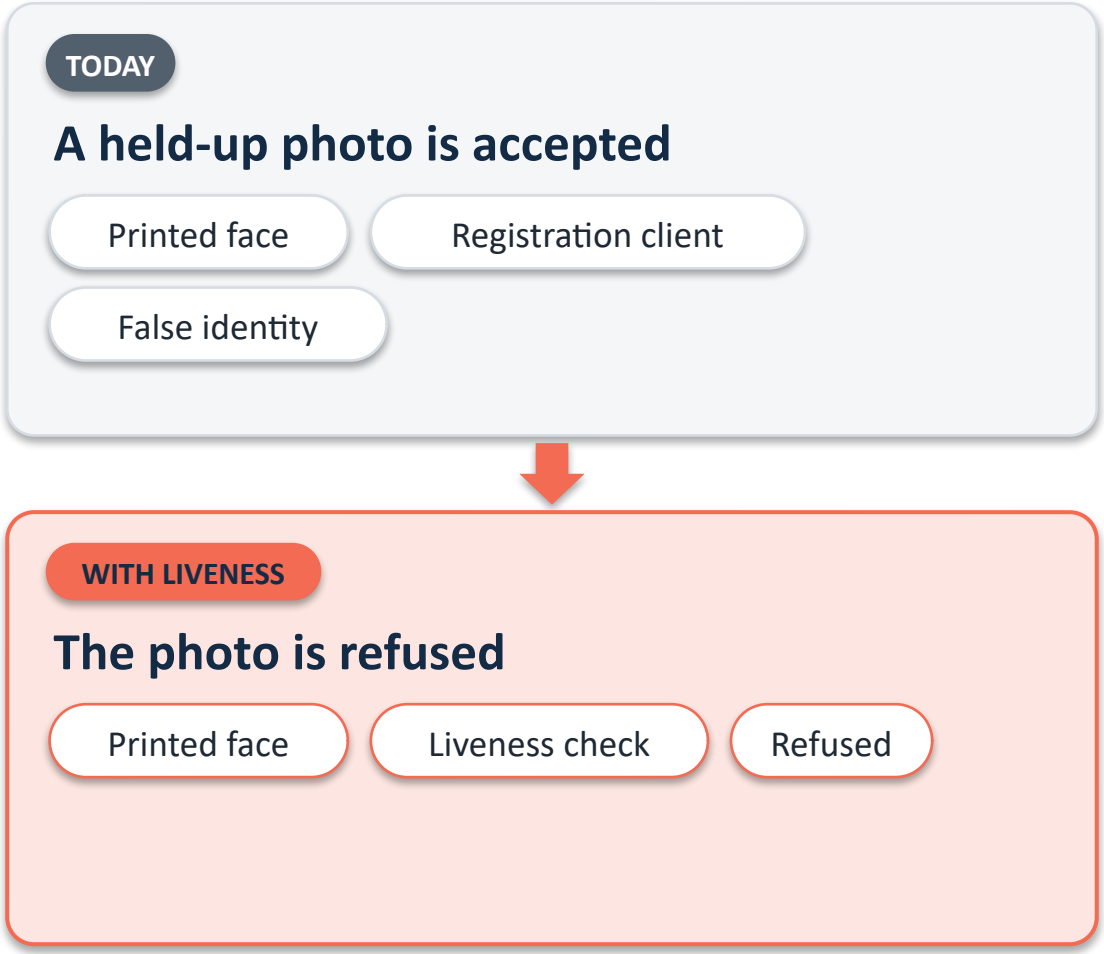
Registration clients accept a held-up photo

1. **Registration**
   a printed or replayed face creates a false identity

2. **Operator login and packet sign-off**
   someone else's photo signs the packet

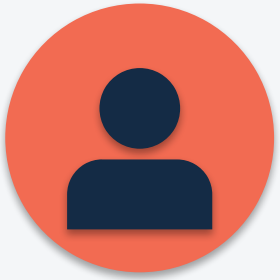
3. **Supervisor approval**
   a photo lets an operator approve their own packets

4. **No liveness check in either client**
   centres often work offline



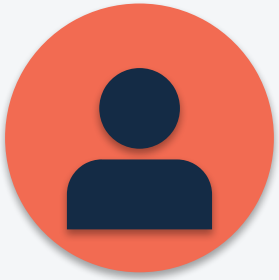
TARGET AUDIENCE

Protects residents, officers and supervisors alike



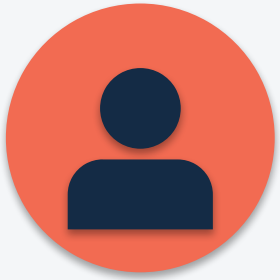
Residents

whose identity can be enrolled from a photo



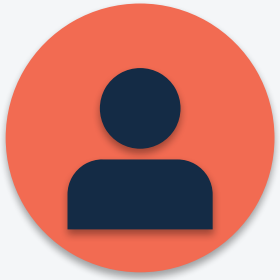
Registration officers and supervisors

whose sign-in can be faked



Programme administrators

who need to know how a face was checked



Device vendors

who need one neutral interface

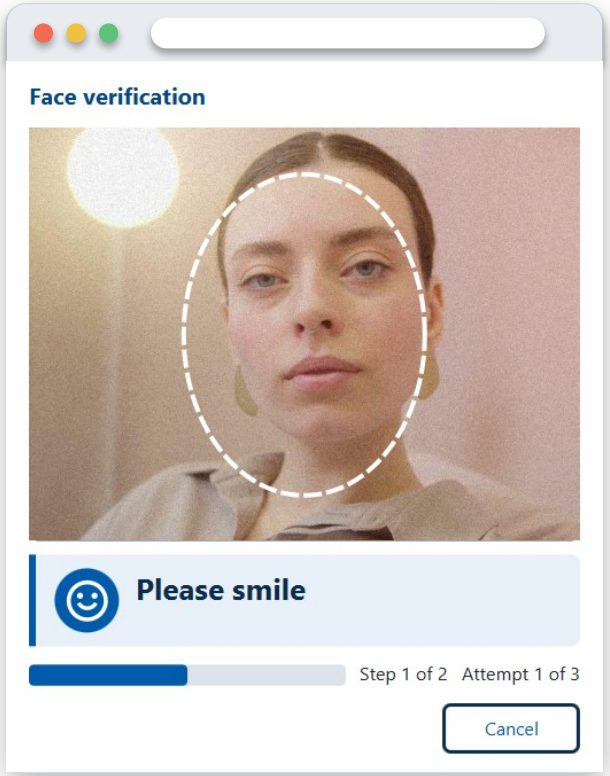
A held-up photo is refused on both clients

1. **One offline engine**
   the same Java 11 library on desktop and Android

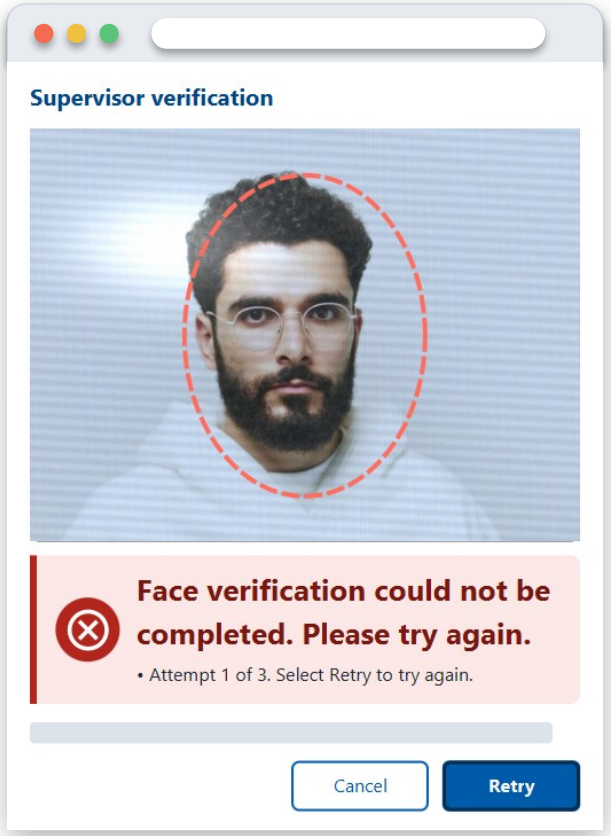
2. **Passive check, then two random challenges**
   chosen from 8 challenge types

3. **Cannot be skipped**
   no face capture without a pass

4. **Six workflows**
   from registration to supervisor approval



Challenge: Please smile



Supervisor replay refused

ARCHITECTURE

One shared engine behind both clients

Outside the clients

- **Face devices, L0 / L1** — SBI device service or tablet camera
- **Signed model bundle** — verified at start-up
- **MOSIP server** — settings sync and packets, needs a connection

↓ preview frames and signed captures

The two MOSIP Registration Clients

- **Desktop client** — JavaFX screens and liveness services
- **Android client** — Flutter panels and livenessmanager

↓ both call one engine contract

Shared engine: Java 11, the same jar on both clients

- **Engine contract** — engine, session, device, config, audit
- **Default engine** — passive PAD, then active challenges
- **ONNX models** — YuNet, Face Mesh, MiniFASNet, SFace
- **Face match SDK** — 1:1 face match

The engine sits behind LivenessEngineProvider, so a certified vendor engine can replace it.

TECHNOLOGY STACK

Java and ONNX models run the whole check offline

01 Languages

- Java 11: engine and desktop client
- Java and Dart (Flutter 3.10.4): Android client
- Python 3: evaluation tools

02 Frameworks and models

- JavaFX desktop, CameraX on Android
- ONNX Runtime 1.20.0 on both clients
- YuNet, Face Mesh, MiniFASNet, SFace

03 MOSIP

- Registration Client, desktop and Android
- SBI face devices, MockMDS
- Sync settings, CBEFF packet, audit, IBioApiV2

04 Build and infrastructure

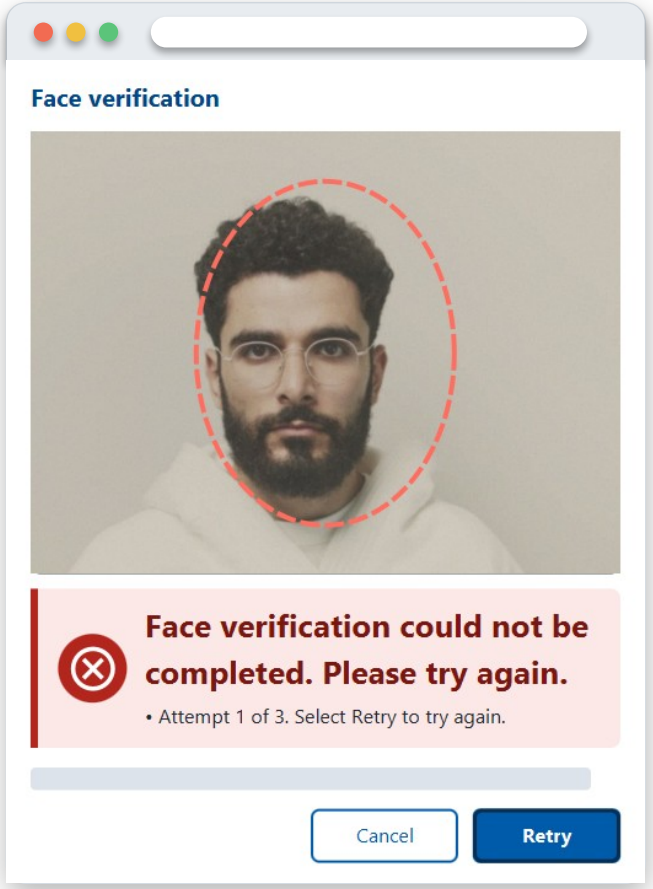
- Maven (JDK 11), Gradle (JDK 21)
- Signed model bundle, RSA-3072 manifest
- pytest and JUnit test suites

DEMO

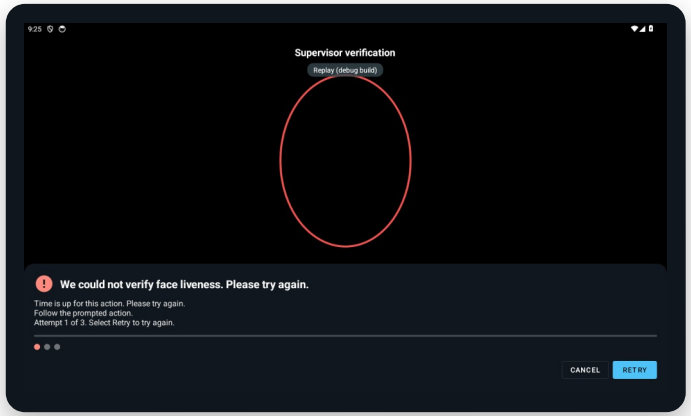
Print refused, replay refused, same decision on Android

1. **Resident capture**
   passive check, then Please smile
2. **A print**
   refused with a generic message
3. **A replay**
   refused at supervisor approval
4. **Policy by sync**
   a server setting reaches the client
5. **Android**
   same engine, same messages
6. **Evidence**
   LIVENESS_* entries in the packet

Demo video:
https://youtu.be/DlSre35yWCo



Desktop: print refused



Android: replay refused

EVIDENCE

0 of 226 synthetic attacks and 0 of 32 real attacks accepted

2,707

tests passed, 0 failures

run of 2026-10-25

0 / 226

synthetic attacks accepted

shipped policy, 275 synthetic transactions

0 / 32

real attacks accepted

print, phone photo, video replay

8 / 8

real genuine accepted

phone camera, two team members

Also measured: 71 / 71 desktop end-to-end checks (2026-10-20, two runs, shipped policy) · 0 / 40 synthetic genuine rejected (registration 0 of 28, sign-in 0 of 12) · 6 / 6 on-phone tests (OnePlus CPH2569, Android 15) · 58-71 ms phone per-frame median (2 threads, CPU or XNNPACK, two runs) · Real-face run 2026-10-24: the phone photo reached passive PAD and was refused 4 of 4; the other 28 attack transactions ended at the capture-quality gate before any decision. Metrics follow ISO/IEC 30107-3.

REQUIREMENTS COVERED

All ten mandatory task groups are built; hardware gaps are marked

Mandatory tasks

1 L0/L1 devices	docs/traceability.md 7.1
2 Passive liveness	docs/test-report/README.md
3 PAD	docs/eval/results/
4 Active liveness	test-report/desktop-e2e.md
5 Resident registration	test-report/desktop-e2e.md
6 Operator login	test-report/android-e2e.md
7 Supervisor sign-in	test-report/desktop-e2e.md
8 Configuration	docs/traceability.md 7.8
9 Error handling	docs/traceability.md 7.9
10 UI/UX	docs/ux/README.md

Bonus tasks

B1 Mock L0/L1 device	devices/mock-mds/
B2 Liveness scenarios	eval/scenarios/
B3 PAD attack scenarios	eval/results/20261014-shipped/
B4 Interoperability tests	test-report/interoperability.md
B5 Model versions	models/manifest.json
B6 Signed offline updates	e2e-15-model-import-verified.png
B7 Low-resource Android	android-performance.md
B8 Hardware acceleration	android-performance.md

Tick: done, with the file shown. Dash: built, a hardware or volunteer run still open (see Scope). Evidence files are in docs/test-report/ and docs/eval/. Good-to-have G1 to G9: 6 done, 3 partial (docs/traceability.md 9). Source: docs/traceability.md.

IMPACT

Six workflows protected in each client

Who benefits



Residents
an identity cannot be enrolled from a photo



Supervisors
approval needs a live face



Administrators
evidence in every packet

What changes, counted

0 / 226

synthetic attacks accepted

100 of 226 with the passive-first flow

0 / 32

real attacks accepted

print, phone photo, video replay

8 / 8

real genuine accepted

phone camera, two team members

6

workflows protected

in each client

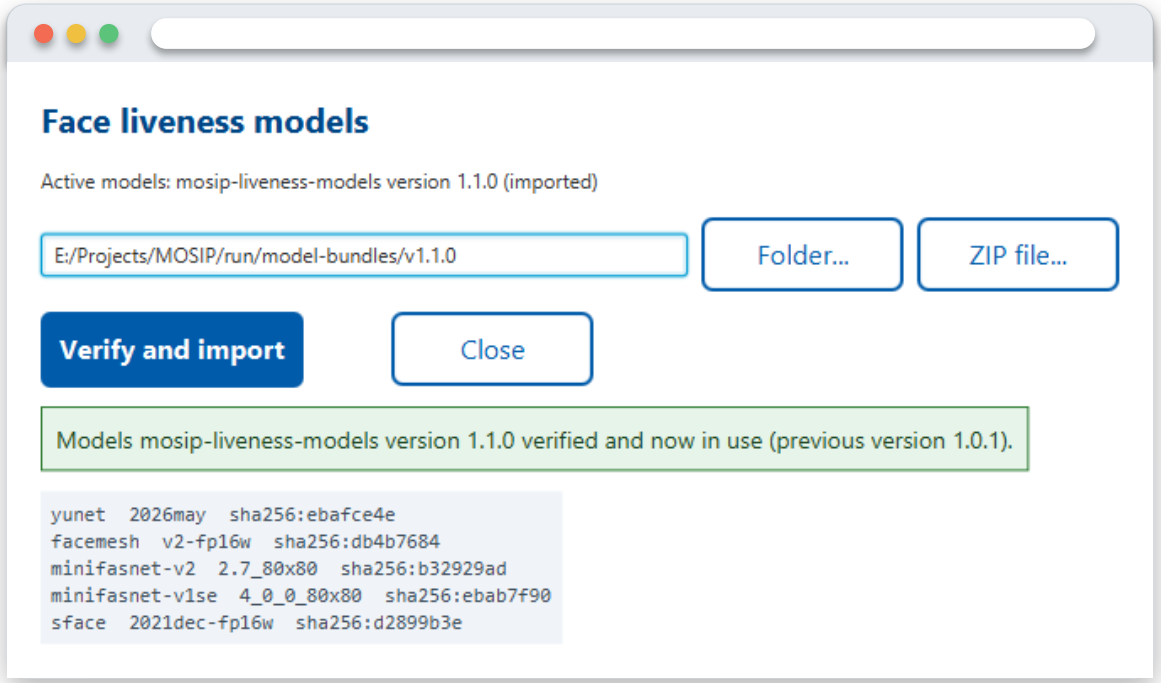
A pass unlocks one signed capture of the same face

1. **Capture binding**
   a pass unlocks one signed capture of the same face

2. **Built on MOSIP's own parts**
   SBI, sync, CBEFF packet, audit, biometric SDK

3. **Same decision on both clients**
   replayed cases match on desktop and Android

4. **Signed model bundle**
   SHA-256 per file, RSA-3072 signed manifest



Signed model bundle verified at import

Scope and next steps

Scope

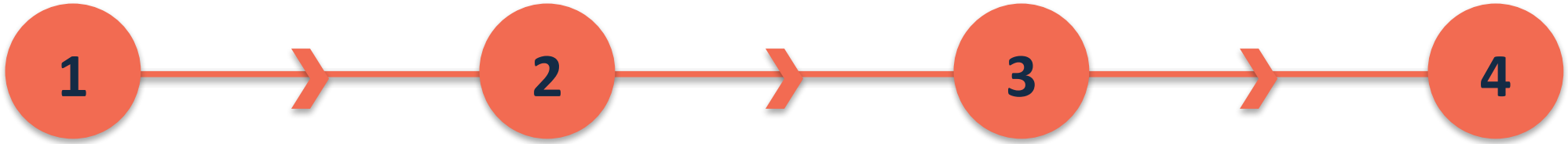
- Real-face run: 2 team members in normal light; dim light and more people follow the in-house protocol
- Per-frame time is above the 20 ms target: p95 127.7 ms on a shared laptop, p50 58-71 ms on the phone
- Laptop camera: 2 of 4 genuine transactions accepted; the clips hold a short natural blink

Runs still to do

- Volunteer evaluation: 12 people, 8 attack types
- Vendor SBI face devices and L1 on Android
- The screen-pattern check against adaptive screen replay is proven on software-made screens

ROADMAP

Next: a volunteer evaluation and vendor devices



Volunteer evaluation

12 people, 8 attack types, ISO/IEC 30107-3 method

Android vendor devices

vendor SBI face devices and L1

Device-side PAD for L1 devices

to stop adaptive replays

Upstream

the changes as reviewable pull requests

Team Badmosip

KB

Kanav Bhardwaj

IB

Ivan Bhargava

NM

Nikunj Mahajan

IS

Ishaan Sodhi

HM

Harsh Modi

Repository

https://github.com/Paradox-73/Face_Liveness_Detection_and_Presentation_Attack_Detection

Public on GitHub, with the code, the test reports and the screenshots behind every number in this deck

TRY IT

Run it yourself

```
$ bash scripts/test-all.sh
```

You need JDK 11 with JavaFX (JAVA11_HOME) and JDK 21 (JAVA21_HOME); the Maven mirror settings and steps are in docs/install/README.md.

You see One command builds and tests the engine, MockSync, MockMDS, the scenario runner and the Python evaluation tools, and stops at the first failed step. --only engine,mocksync,mockmds,runner,eval picks steps.

Scan for the repository

Repository **https://github.com/Paradox-73/Face_Liveness_Detection_and_Presentation_Attack_Detection**